

CSC-220: Neural Networks and Deep Learning

Lecture 01 - DL: Mathematical Building Blocks¹

September 8th, 2026

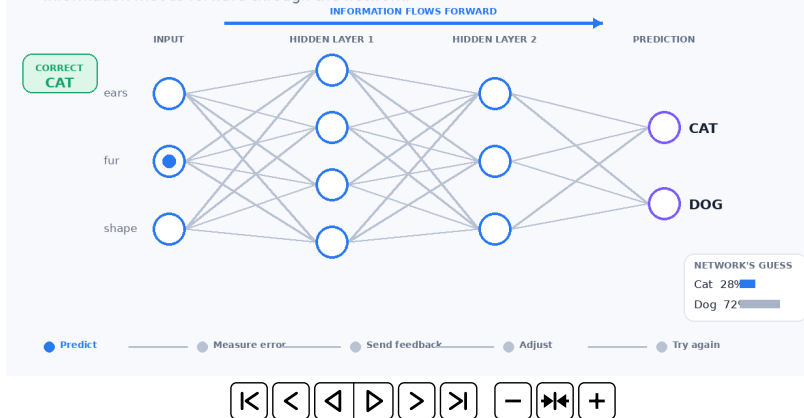
¹Image content: https://deeplearningwithpython.io/chapters/chapter02_mathematical-building-blocks/

How Learning (Backpropagation) Works

HOW A NEURAL NETWORK LEARNS

1. Make a prediction

Information moves forward through the network.



How Learning (Backpropagation) Works

What are the “ingredients” of learning in neural networks?

How Learning (Backpropagation) Works

What are the “ingredients” of learning in neural networks?

A proper *data representation* that can be processed by the model.

How Learning (Backpropagation) Works

What are the “ingredients” of learning in neural networks?

- A proper *data representation* that can be processed by the model.

- A proper *model architecture* that can represent the underlying patterns in the data.

How Learning (Backpropagation) Works

What are the “ingredients” of learning in neural networks?

- A proper *data representation* that can be processed by the model.

- A proper *model architecture* that can represent the underlying patterns in the data.

- A *loss function* that measures how well the model is performing.

How Learning (Backpropagation) Works

What are the “ingredients” of learning in neural networks?

A proper *data representation* that can be processed by the model.

A proper *model architecture* that can represent the underlying patterns in the data.

A *loss function* that measures how well the model is performing.

A method to compute the *gradient* of the loss function with respect to the model's parameters.

How Learning (Backpropagation) Works

What are the “ingredients” of learning in neural networks?

A proper *data representation* that can be processed by the model.

A proper *model architecture* that can represent the underlying patterns in the data.

A *loss function* that measures how well the model is performing.

A method to compute the *gradient* of the loss function with respect to the model's parameters.

An *optimization algorithm* that uses the gradient to update the model's parameters in order to minimize the loss.

Data Representations for Neural Networks

Modern machine learning systems represent data as *tensors* (multidimensional arrays).

A tensor is a container for numerical data:

Scalars (0D), vectors (1D), matrices (2D), and higher-dimensional arrays

Tensors generalize matrices to any number of dimensions (called *axes*). They are the fundamental building blocks of deep learning frameworks like TensorFlow.

Working with tensors is essential — they are the core data structure in all machine learning code.

Scalars (Rank-0 Tensors)

A *scalar* is a tensor containing a single number (0D tensor).
In NumPy, scalar tensors have `ndim = 0`.

```
1  import numpy as np
2  x = np.array(12)
3
4  print(x)          # array(12)
5  print(x.ndim)     # 0
```

Vectors (Rank-1 Tensors)

A *vector* is a rank-1 tensor (1D tensor) with exactly one axis.

The number of entries determines its dimensionality (e.g., 5 entries $\rightarrow$ 5D vector).

The number of axes is called the *rank*.

```
1  import numpy as np
2  x = np.array([12, 3, 6, 14, 7])
3
4  print(x)           # [12  3  6 14  7]
5  print(x.ndim)      # 1
```

Matrices (Rank-2 Tensors)

A *matrix* is a rank-2 tensor (2D tensor) with two axes.

These axes are typically called *rows* and *columns*.

A matrix can be seen as a rectangular grid of numbers.

```
1 import numpy as np
2 x = np.array([[5, 78, 2, 34, 0],
3               [6, 79, 3, 35, 1],
4               [7, 80, 4, 36, 2]])
5
6 print(x.ndim)  # 2
```

First row: [5, 78, 2, 34, 0] First column: [5, 6, 7]

Rank-3 Tensors and Higher

A *rank-3 tensor* (3D tensor) is obtained by stacking matrices.

It can be visualized as a cube of numbers.

Higher-rank tensors are formed by stacking lower-rank tensors.

```
1  import numpy as np
2  x = np.array([
3      [[5, 78, 2, 34, 0],
4       [6, 79, 3, 35, 1],
5       [7, 80, 4, 36, 2]],
6
7      [[5, 78, 2, 34, 0],
8       [6, 79, 3, 35, 1],
9       [7, 80, 4, 36, 2]]
10 ])
11
12 print(x.ndim)    # 3
```

In practice, most deep learning uses tensors of rank 0–4 (up to 5 for video data).

Key Attributes of Tensors

Key Attributes of Tensors:

Number of axes (rank) — Defines how many dimensions a tensor has (e.g., matrix = 2, rank-3 tensor = 3). In Python: `ndim`.

Key Attributes of Tensors

Key Attributes of Tensors:

Number of axes (rank) — Defines how many dimensions a tensor has (e.g., matrix = 2, rank-3 tensor = 3). In Python: `ndim`.

Shape — Tuple describing size along each axis (e.g., matrix: (3, 5), tensor: (3, 3, 5), vector: (5,), scalar: ()).

Key Attributes of Tensors

Key Attributes of Tensors:

Number of axes (rank) — Defines how many dimensions a tensor has (e.g., matrix = 2, rank-3 tensor = 3). In Python: `ndim`.

Shape — Tuple describing size along each axis (e.g., matrix: (3, 5), tensor: (3, 3, 5), vector: (5,), scalar: ()).

Data type (dtype) — Type of values stored in the tensor (e.g., `float32`, `float64`, `int32`, `bool`).

Tensor Slicing in NumPy

Selecting parts of a tensor is called *slicing*.

`start:stop` selects a range along an axis.

`:` selects the entire axis.

```
1  # Basic slicing
2  my_slice = train_images[10:100]
3  print(my_slice.shape)      # (90, 28, 28)
4
5  # Equivalent forms
6  train_images[10:100, :, :]
7  train_images[10:100, 0:28, 0:28]
```

Data Batches

The first axis (axis 0) is usually the *samples axis*.

Deep learning models process data in small groups called *batches*.

Each batch has a fixed size (e.g., 128 samples).

```
1  # First batch
2  batch = train_images[:128]
3
4  # Next batch
5  batch = train_images[128:256]
6
7  # n-th batch
8  n = 3
9  batch = train_images[128*n : 128*(n+1)]
```

Axis 0 is often called the *batch axis*.

Data Batches

The first axis (axis 0) is usually the *samples axis*.

Deep learning models process data in small groups called *batches*.

Each batch has a fixed size (e.g., 128 samples).

```
1  # First batch
2  batch = train_images[:128]
3
4  # Next batch
5  batch = train_images[128:256]
6
7  # n-th batch
8  n = 3
9  batch = train_images[128*n : 128*(n+1)]
```

Axis 0 is often called the *batch axis*.

Image Data as Tensors

Images are typically represented with three dimensions: *height*, *width*, and *color channels*.

Grayscale images have one channel, color images have three (RGB).

By convention, images are stored as rank-3 tensors:

Grayscale: (height, width, 1)

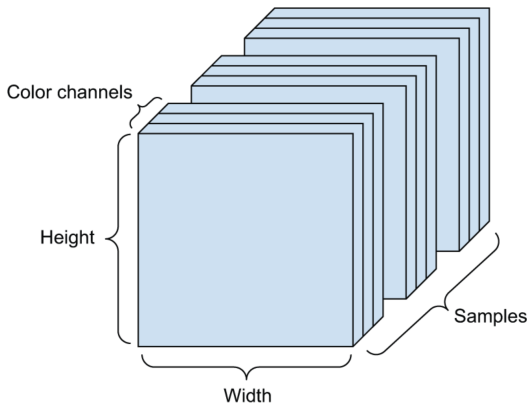
Color: (height, width, 3)

A batch of images adds one more dimension (batch size):

Grayscale batch: (128, 256, 256, 1)

Color batch: (128, 256, 256, 3)

A rank-4 image data tensor



Tensor Operations

Just like programs reduce to simple binary operations (AND, OR, NOR), deep learning reduces to a small set of *tensor operations*.

These operations (or *tensor functions*) are applied to numerical data.

Common examples:

- Addition

- Multiplication

- Other element-wise transformations

Element-wise Operations

Element-wise operations are applied independently to each entry of a tensor.

Examples: addition, ReLU ($\max(x, 0)$).

These operations are highly parallelizable (vectorized).

```
1 def naive_relu(x):  
2     x = x.copy()  
3     for i in range(x.shape[0]):  
4         for j in range(x.shape[1]):  
5             x[i, j] = max(x[i, j], 0)  
6     return x
```

In practice, this is done without loops using vectorized operations.

Element-wise Addition

Addition can also be applied element-wise to tensors.

Both tensors must have the same shape.

Like ReLU, this operation is naturally parallelizable.

```
1  def naive_add(x, y):  
2      x = x.copy()  
3      for i in range(x.shape[0]):  
4          for j in range(x.shape[1]):  
5              x[i, j] += y[i, j]  
6      return x
```

In practice: $x + y$ (vectorized, no loops needed).

Broadcasting

Broadcasting allows operations on tensors with different shapes.
The smaller tensor is automatically expanded to match the larger one.

How it works:

Add missing axes to the smaller tensor

Repeat values along those axes to match the shape

```
1  import numpy as np
2
3  x = np.random.random((32, 10))  # matrix
4  y = np.random.random((10,))     # vector
5
6  z = x + y  # broadcasting
```

Example: $(32, 10) + (10,) \rightarrow (32, 10)$

Broadcasting (Implementation Insight)

Broadcasting does **not** actually copy data in memory.

The expansion is *virtual* (done at the algorithm level).

Still, it's useful to think of the smaller tensor as being repeated.

```
1  def naive_add_matrix_and_vector(x, y):  
2      x = x.copy()  
3      for i in range(x.shape[0]):  
4          for j in range(x.shape[1]):  
5              x[i, j] += y[j]  
6      return x
```

In practice: $x + y$ uses broadcasting (no loops, no copies).

Tensor Product (Dot Product)

The *tensor product* (dot product / matmul) is a fundamental operation. It combines tensors through multiplication and summation.

```
1  import numpy as np
2
3  x = np.random.random((32,))
4  y = np.random.random((32,))
5
6  z = np.matmul(x, y)
7  # equivalent:
8  z = x @ y
```

Mathematical notation: $z = x \cdot y$

Dot Product (Vector $\times$ Vector)

The dot product multiplies two vectors and sums the results.

The result is a *scalar*.

Vectors must have the same length.

```
1  def naive_vector_product(x, y):  
2      z = 0.0  
3      for i in range(x.shape[0]):  
4          z += x[i] * y[i]  
5      return z
```

Mathematical form: $z = \sum_i x_i y_i$

Matrix $\times$ Vector Product

Multiplying a matrix X by a vector y returns a vector.

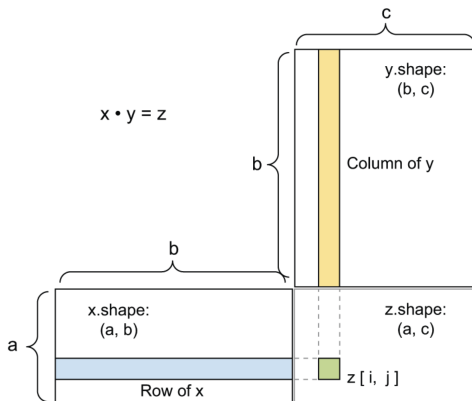
Each output element is the dot product of a row of X with y .

```
1  def naive_matrix_vector_product(x, y):  
2      z = np.zeros(x.shape[0])  
3      for i in range(x.shape[0]):  
4          for j in range(x.shape[1]):  
5              z[i] += x[i, j] * y[j]  
6      return z
```

Equivalent view: $z[i] = \text{dot}(x[i, :], y)$

Not symmetric: $\text{matmul}(x, y) \neq \text{matmul}(y, x)$

A schematic representation of tensor products



Tensor Reshaping

Reshaping changes the shape of a tensor without changing its data.
The total number of elements must remain the same.

```
1  import numpy as np
2
3  x = np.array([[0, 1],
4                [2, 3],
5                [4, 5]])    # shape (3, 2)
6
7  x = x.reshape((6, 1))
8  # shape (6, 1)
9
10 x = x.reshape((2, 3))
11 # shape (2, 3)
```

Same data, different shapes.

The Engine of Neural Networks

Each layer transforms input as:

$$\text{output} = \text{relu}(\text{matmul}(\text{input}, W) + b)$$

W , b are *trainable parameters* (weights and bias)

Initially random $\rightarrow$ produce meaningless outputs

Learning = adjusting these parameters

How Training Works

Sample a batch: x, y_{true}

Compute predictions: y_{pred}

Compute loss (error)

Update weights to reduce loss

Goal: minimize difference between y_{pred} and y_{true}

Why Not Try All Values?

Idea: tweak one weight, check if loss improves

Repeat for all weights

Works in theory...

But extremely slow (millions/billions of parameters)

We need a smarter way → Gradient Descent

Gradient Descent Intuition

Model functions are *smooth* (differentiable)

Small change in weights $\rightarrow$ small change in loss

Gradient tells us:

- Which direction reduces the loss

- How much to adjust

Update all weights at once in the right direction

What's a Derivative?

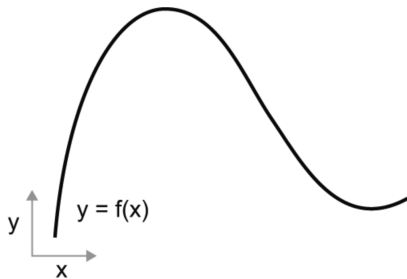
A function maps input to output: $f(x) = y$

The derivative measures:

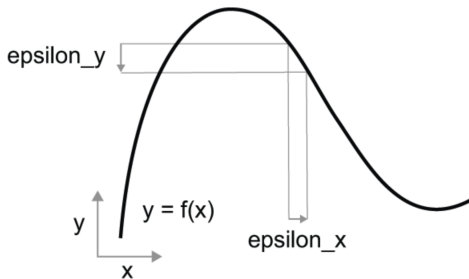
How much y changes when x changes

Derivative = slope = rate of change

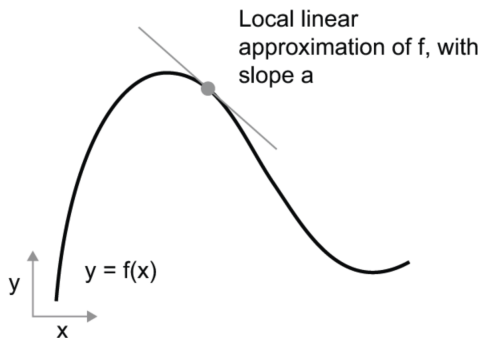
Gradient Descent: visual intuition



Gradient Descent: visual intuition



Gradient Descent: visual intuition



Gradient: Derivative of Tensor Functions

A function can map:

Scalar $\rightarrow$ scalar $\rightarrow$ curve

Vector $\rightarrow$ scalar $\rightarrow$ surface

Tensor $\rightarrow$ scalar $\rightarrow$ higher-dimensional surface

The *gradient* is the generalization of the derivative.

It tells us how the output changes when inputs change.

Derivative = slope (1D)

Gradient = direction of steepest change (multi-D)

Stochastic Gradient Descent (SGD)

In theory: find minimum by solving $\nabla f(W) = 0$

In practice: impossible for large models (millions of parameters)

Instead: iteratively update parameters using gradients

Move weights in the direction that reduces the loss

$$W = W - \text{learning_rate} * \text{gradient}$$

SGD Algorithm (Mini-batch)

Sample a batch: x, y_{true}

Forward pass $\rightarrow$ compute y_{pred}

Compute loss

Backward pass $\rightarrow$ compute gradient

Update weights

learning_rate controls step size

Key Ideas Behind SGD

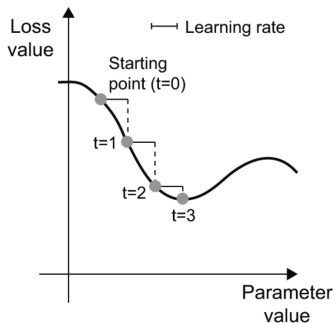
Uses random batches (stochastic = random)

Efficient for large datasets

Updates happen step-by-step (not all at once)

Gradually reduces loss over time

Stochastic Gradient Descent (SGD) Intuition



Stochastic Gradient Descent (SGD) Intuition

